AI Usage Log

Platform: ChatGPT

Model: GPT-6

Note: This log records the actual Lab 8 interactions available in the conversation. Assistant summaries emphasize the setup, testing, and programming concepts discussed; user prompts have not been rewritten as questions that were never asked.

Interaction 1

User

EECS 348: Software Engineering - Fall 2026

Lab 8 - More C++ Programming: String-Based Calculator

Objective: In this lab, you will implement string-based arithmetic operations in C++ without converting strings to built-in numeric types. You will practice parsing, validation, and manual arithmetic on numbers represented as strings. Specifically, you will check whether a given string is a valid double-precision number format, and then implement addition (and optionally, multiplication) between such numbers.

What to turn in: Please provide a URL to your GitHub repository. If you are unable to push your code to GitHub, you may instead use KU GitLab, which works in a very similar way. Your repository must be private so that your work is not publicly searchable. Limit access to yourself, the course instructor, and the course GTAs. Make sure your GTA has access to review and grade your submission.

(In your repo, go to Settings → Collaborators (under “Access”), click Add people, type their email, choose the right match from the list, and confirm. They’ll get an invitation email, and they get access once they accept it.)

Grading:

- A valid Makefile must be included in the repository. [5 points]

- The program reads the test cases from the file. [5 points]
- There are 30 test cases, and each one is 3 points. [90 points]

(Grading is based on correct arithmetic results and input validation. Prompt wording and equivalent numeric formatting, such as 13 and 13.0, will not affect your grade.)

Programming problem #1. Basic Calculator

1. Read the candidate numbers from the file:

Implement a function to load string-formatted numbers from a user-specified file, where each line in the file contains two such numbers. (Each line must contain exactly two whitespace-separated values. If a line contains fewer or more than two values, print an error message and continue to the next line.)

Here are some samples:

123 456

123.456 +1

+123 -123

... (there are thirty lines)

2. Identify if a number is a valid double number:

For each line in the file, check whether the string represents a valid double number. If both are valid, proceed to execute the subsequent functions. Otherwise, print a message indicating that the number is invalid and continue processing the next line.

A valid double number follows the format (+|-)A(.B), where:

- An optional '+' or '-' sign may appear at the beginning;
- A sequence of digits follows, which may include leading zeros (these can be removed before calculation for simplicity);
- An optional decimal point may appear; if it does, there must be at least one digit on both sides of it.

Some valid double examples: 1, 1.0, +1.0, +0001.0, -0001.005

Some invalid double examples: A, +-1, -5., -5.-5

3. Perform addition between the parsed double numbers:
If both numbers are valid, sum them up and print the result.

help me with this lab. where should I begin and what steps should i take to successfully finish? I don't want the answers, but i need help learning how to do everything. Guide/teach me

Assistant Summary

Generated a C++ calculator project template. Described the implementation as using manual digit-by-digit addition/subtraction, sign handling, decimal alignment, and input validation rather than converting operands to floating-point numbers. Provided step-by-step instructions for compiling, testing, and submitting through a private repository, while providing context for each instruction.

Interaction 2

User

what does if (!validFirst || !validSecond) {
cout << "ERROR: invalid number";
if (!validFirst) cout << " \"" << first << "\"";
if (!validSecond) cout << " \"" << second << "\"";
cout << '\n';
continue;
why do i need so many <<, im still confused

Assistant Summary

Explained how the conditional statement checks whether either input number is invalid using the NOT (`!`) and OR (`||`) operators. Clarified how `cout` and multiple `<<` operators print error messages, including quotation marks around invalid inputs. Also explained how `continue` skips the current test case and moves to the next line, and demonstrated a simpler version of the code.

Interaction 3

User

Why can't I just use stod() to convert the strings into doubles?

Assistant Summary

Explained how `stod()` converts a string into a double and why it cannot be used for this lab. Discussed the assignment's requirement to perform arithmetic using strings, the precision limitations of doubles, and how manual digit-by-digit addition with carry handling works as an alternative.

Interaction 4

User

<screenshot>what to do from here

Assistant Summary

Explained how to transfer the file to the KU Cycle server, extract it, navigate to the project directory, compile with `make`, run the test target, and compare program output with `expected_output.txt`.

Interaction 5

User

how do I make sure the program rejects inputs like +-1, 5., and 2.3.4 or smth else like that

Assistant Summary

Explained how to validate number strings by checking each character for valid digits, optional signs, and decimal points. Demonstrated how the validation function rejects multiple signs, missing decimal digits, letters, and multiple decimal points. Suggested creating a separate test file containing invalid and valid inputs to verify error handling.

Interaction 6

User

how does substr() work when removing leading zeros, still confused about that

Assistant Summary

Explained how `substr()` extracts part of a string starting at a specified index and how `find_first_not_of('0')` identifies the first nonzero digit. Walked through removing leading zeros from `"000123"` and explained how `string::npos` and the ternary operator handle strings containing only zeros. Clarified why zeros within a number must be preserved.